ChatGPT

Milestone 3 Plan

You

AgroSphere — Project Context for Milestone 3

We are developing AgroSphere, an AI-powered Rubber Plantation Management and Digital Twin platform using OpenAI Codex in VS Code.

Architecture

- Microservices architecture using Domain-Driven Design.
- Multi-repository architecture: each service has an independent Git repository, build, test suite, and CI/CD pipeline.
- Java 21, Spring Boot for business microservices.
- Python/FastAPI for telemetry and AI services.
- PostgreSQL/PostGIS for transactional and geospatial data.
- RabbitMQ for asynchronous messaging.
- MQTT for IoT communication.
- Keycloak for OIDC authentication.
- API Gateway for client-facing APIs.
- Angular + CesiumJS for the 3D web digital twin.
- Unreal Engine + Cesium for Unreal for future immersive experiences.
- Docker Compose for local integration.
- OpenAPI and versioned event contracts.
- C4 architecture diagrams in PlantUML.

Repositories

- agrosphere-architecture

Take your curiosity further
Scan to get the ChatGPT app

- agrosphere-contracts
- agrosphere-infrastructure
- agrosphere-api-gateway
- agrosphere-estate-service
- agrosphere-spatial-service
- agrosphere-operations-service
- agrosphere-telemetry-service
- agrosphere-analytics-service
- agrosphere-ai-service
- agrosphere-web
- agrosphere-immersive

Completed Milestones

- Milestone 0: Multi-repository foundation.
- Milestone 1A: Estate Management Service.
- Milestone 1B: Spatial Service with PostGIS.
- Milestone 1C: API Gateway, Keycloak, RabbitMQ, integration infrastructure.
- Milestone 2: Angular + CesiumJS Rubber Plantation Digital Twin.

These milestones have been reported as completed by Codex. Actual implementation details and validation results should be verified from the repositories.

Next Milestone

Milestone 3 — Rubber Plantation Operations, IoT Telemetry, and Analytics.
Expected capabilities:

1. Rubber tapping schedules and records.
2. Latex collection and weighment management.
3. Dry Rubber Content (DRC) calculations.
4. Crop-health inspections.

Take your curiosity further
Scan to get the ChatGPT app

5. Simulated IoT sensors and MQTT ingestion.
6. Weather and soil moisture monitoring.
7. RabbitMQ domain events.
8. Operational KPI projections.
9. Spatial KPI overlays in CesiumJS.
10. Real-time updates and alerts.

Important Instructions

I want detailed, implementation-ready Codex prompts.
Each milestone should specify:

- Functional and nonfunctional requirements.
- Domain models.
- Service responsibilities.
- Database ownership.
- REST and event contracts.
- Security.
- Automated testing.
- Docker/CI integration.
- Definition of Done.
- Completion reporting.

Maintain strict multi-repository microservices boundaries.
Do not introduce a monolith or direct cross-service database access.
Do not assume APIs or functionality exist without verifying the actual repository implementation.
Please recommend how to divide Milestone 3 into smaller independently executable Codex prompts, then provide the first prompt.

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

AgroSphere — Milestone 3 Implementation Plan

Milestone 3: Rubber Plantation Operations, IoT Telemetry, and Analytics

I recommend dividing Milestone 3 into **seven independently executable Codex prompts (3A–3G)**, each producing a testable increment of the platform.

The objective is to move AgroSphere from a 3D visualization of a rubber plantation to an operational digital twin that reflects plantation activities, production, environmental conditions, and alerts.

1. Recommended milestone breakdown

MILESTONE 3 — EXECUTION ROADMAP

1 **3A — Rubber Tapping Operations**

Start here

Tapping assignments, schedules, execution records, missed tapping, worker productivity, and audit history.

Primary repository: agrosphere-operations-service

2 **3B — Latex Collection, Weighment and DRC**

Latex collection batches, collection points, weighing, DRC measurements, dry-rubber yield, and production reconciliation.

Primary repository: agrosphere-operations-service

3 **3C — Crop Health and Field Inspections**

Disease observations, severity, tree/block inspections, treatment recommendations, and follow-up activities.

Primary repository: agrosphere-operations-service

4 **3D — IoT Telemetry and Sensor Simulation**

Take your curiosity further
Scan to get the ChatGPT app

MQTT broker integration, sensor registration, simulated environmental readings, ingestion pipelines, and persisted time-series data.
Primary repository: agrosphere-telemetry-service

5 **3E — Operational Analytics and KPI Projections**
Aggregated tapping performance, rubber yield, DRC trends, plantation health, weather and soil moisture metrics.
Primary repository: agrosphere-analytics-service

6 **3F — Spatial KPI Overlays and Live Digital Twin**
CesiumJS KPI layers, block colors, sensor markers, production trends, live updates, and operational dashboards.
Primary repository: agrosphere-web

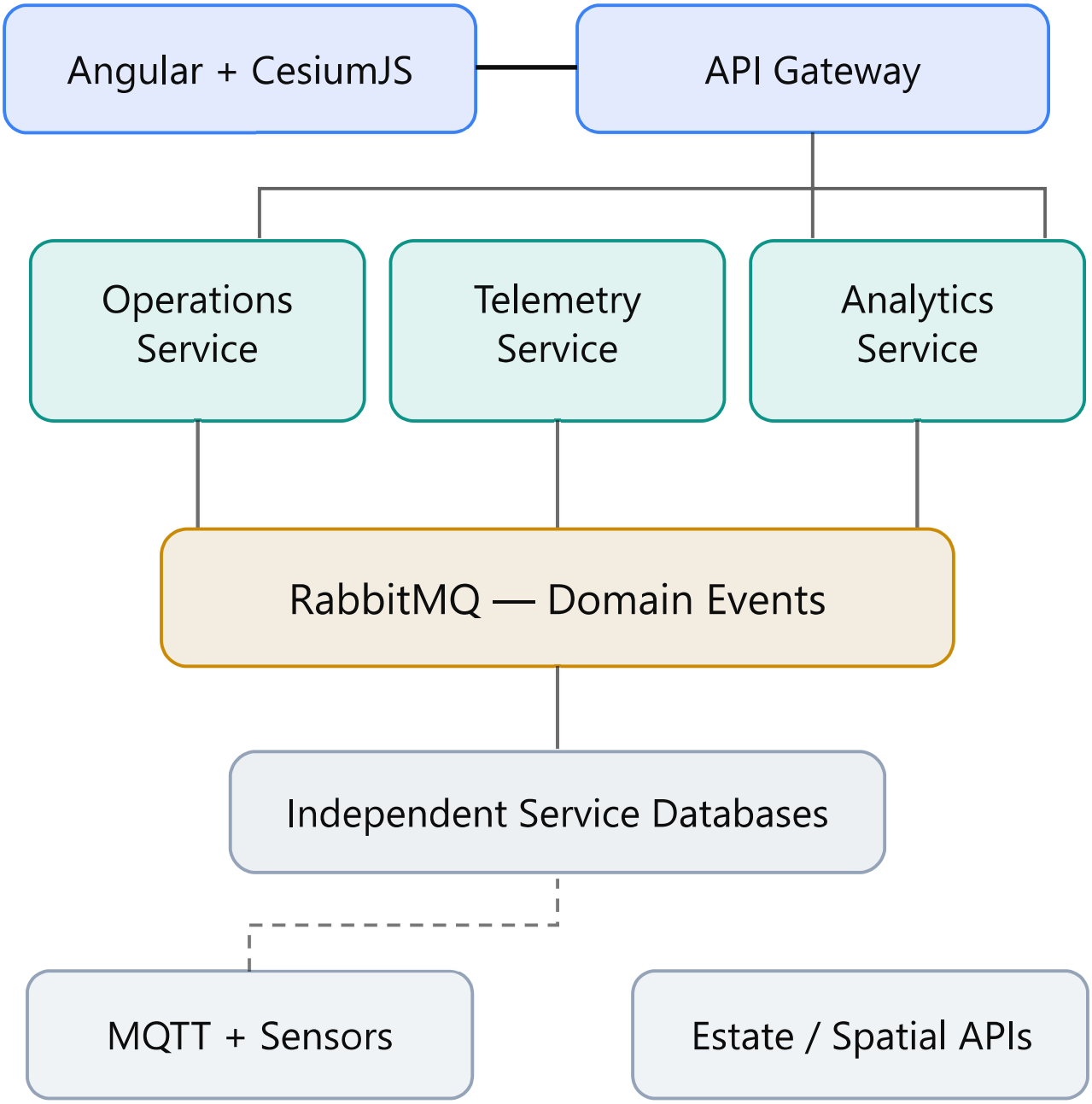
7 **3G — Alerts, Cross-Service Integration and Hardening**
Alert rules, event-driven notifications, system-wide integration tests, resilience verification, CI/CD validation, and documentation.
Cross-repository integration, with alert ownership explicitly assigned

Each increment can be implemented and validated separately, but there are dependencies. Codex should not move to the next increment until the previous increment's contracts and acceptance criteria have been verified.

2. Architecture for Milestone 3

Logical service and event architecture

Take your curiosity further
Scan to get the ChatGPT app



Conceptual overview; connectors are illustrative

Each service owns its data. The operations service publishes events, telemetry ingests MQTT messages, and analytics builds projections from service-owned data and subscribed events. Client-facing APIs are routed through the gateway.

Important design decisions:

Take your curiosity further
Scan to get the ChatGPT app

- **Operations service owns plantation work and production transactions.**
- **Estate service remains the authority for estates, blocks, and other existing plantation master data.**
- **Spatial service remains the authority for geospatial geometry.**
- **Telemetry service owns sensor metadata and time-series measurements.**
- **Analytics service owns derived KPI projections, not operational source transactions.**
- **Contracts repository governs OpenAPI specifications and versioned event schemas.**
- **No service may query another service's database directly.**

Within Milestone 3A, I recommend using a modular monolith *inside the operations microservice*—meaning cohesive internal modules in one independently deployed service, not a monolithic AgroSphere application. There is no need to create a separate microservice for tapping schedules.

3. Milestone dependencies

Milestone	Prerequisites	Main acceptance gate
3A	Verify 1A, 1C	Tapping workflow works end to end
3B	3A	Latex and DRC reconcile correctly
3C	Estate and Spatial contracts	Inspections and findings persist
3D	Infrastructure and MQTT setup	Simulated readings are ingested
3E	3A, 3B, 3C, 3D event contracts	KPIs can be recomputed deterministically
3F	3E, existing Milestone 2	KPIs appear in CesiumJS
3G	3A–3F	Integrated, secured, resilient workflow

3C and 3D may be developed in parallel once their shared contracts are stable. The other increments should follow the dependency order.

Take your curiosity further
Scan to get the ChatGPT app

4. Milestone 3A — Codex implementation prompt

Copy this prompt into Codex in your AgroSphere VS Code workspace.

Copy prompt

Writing

{documentPrompt}

5. Recommended execution and review strategy

I recommend running each Milestone 3 prompt in a separate Codex task, using the same multi-repository VS Code workspace.

Once Codex reports 3A is complete, review its output against the Definition of Done rather than relying on its final summary alone. In particular, verify that RabbitMQ publication is backed by the transactional outbox, that migrations run against an actual PostgreSQL instance, and that worker authorization is enforced by the service.

The key technical principle is that **Milestone 3A must create a dependable operational system of record.** Subsequent milestones will use its domain events and stable APIs to build production tracking, analytics, and eventually the live digital twin.

For Milestone 3B, we can then build latex collection, batch traceability, weighment, and DRC using the real contracts produced by 3A instead of making assumptions about them.

You

Can you please share the prompt for Milestone 3A?

ChatGPT

Below is the complete, implementation-ready **Codex prompt for AgroSphere Milestone 3A — Rubber Tapping Operations**.

Take your curiosity further
Scan to get the ChatGPT app

Open your AgroSphere multi-repository workspace in VS Code and paste this prompt into Codex. It explicitly instructs Codex to inspect existing repositories before implementing changes.

Writing

AgroSphere — Milestone 3A: Rubber Tapping Operations

Role and Objective

You are acting as the **Lead Solution Architect, Senior Java Developer, Domain-Driven Design Specialist, and Integration Engineer** for AgroSphere.

Implement **Milestone 3A — Rubber Tapping Operations**, building on the existing AgroSphere multi-repository microservices platform.

The goal is to implement the complete rubber tapping operational workflow, from creating tapping schedules through recording completed or missed tapping assignments, publishing domain events, and exposing operational summaries.

Implement production-quality, tested code — not only architecture documents or scaffolding.

---

1. Mandatory Operating Instructions

1. Inspect the actual workspace and repositories before making any changes.
2. Do not assume that previously completed milestones have implemented particular APIs, entities, security configurations, or infrastructure capabilities.
3. Preserve strict multi-repository and microservices boundaries.
4. Every microservice must have an independently buildable, testable, and deployable repository.
5. Do not introduce a monolithic AgroSphere application.
6. Do not access another microservice's database directly.
7. Reuse existing conventions, libraries, infrastructure, and domain models wherever appropriate.
8. Do not duplicate estate, block, spatial, or identity master data.
9. Make reasonable implementation decisions independently and document assumptions in Architecture Decision Records.

10. Do not ask routine clarification questions when a safe, reasonable engineering decision is possible.
11. Do not implement Milestone 3B or later features.
12. Do not overwrite unrelated uncommitted changes.
13. Do not push commits or deploy to shared environments without authorization.
14. Never report tests as passing unless they have actually executed successfully.

If a prerequisite is missing, implement a minimal, compatible change in its owning repository where safe. Otherwise document the blocker clearly.

2. Verify the Existing AgroSphere Workspace

Locate and inspect the following repositories:

```
agrosphere-architecture
agrosphere-contracts
agrosphere-infrastructure
agrosphere-api-gateway
agrosphere-estate-service
agrosphere-spatial-service
agrosphere-operations-service
agrosphere-telemetry-service
agrosphere-analytics-service
agrosphere-ai-service
agrosphere-web
agrosphere-immersive
```

For each relevant repository, verify:

- Git root, branch, and uncommitted modifications.
- Build configuration and project structure.
- Existing REST and event contracts.
- Database migration conventions.
- Authentication and authorization patterns.
- RabbitMQ configuration and event naming.
- Docker Compose integration.

- Existing tests and CI workflows.

Pay particular attention to implementations completed during Milestones 1A, 1C, and 2.

Produce:

`agrosphere-architecture/docs/milestones/milestone-3a-discovery.md`

Document verified capabilities, missing prerequisites, architectural decisions, and implementation assumptions.

Continue implementation after discovery unless a critical blocker prevents safe progress.

3. Repository Responsibilities

Primary repository:

`agrosphere-operations-service`

Owns:

- Tapping schedules.
- Tapping assignments.
- Execution records.
- Missed tapping records.
- Tapping productivity calculations.
- Operational audit data.
- Domain event publication.

Supporting repositories:

Repository	Responsibility
`agrosphere-contracts`	OpenAPI specifications and versioned event schemas
`agrosphere-infrastructure`	PostgreSQL, RabbitMQ, Docker Compose, local integration
`agrosphere-api-gateway`	Route configuration, subject to existing conventions

`agrosphere-architecture`	PlantUML diagrams, ADRs, documentation
`agrosphere-estate-service`	Existing authority for estate and block master data

Do not modify unrelated repositories unnecessarily.

Operations must store estate and block identifiers without direct foreign keys into other services' databases.

4. Functional Requirements

FR-01: Tapping Schedule Management

Implement:

- Create tapping schedule.
- Retrieve schedule by ID.
- List and filter schedules.
- Modify draft schedules.
- Activate schedule.
- Suspend schedule.
- Close schedule.
- Generate dated tapping assignments.

Support:

- Estate and block identification.
- Schedule name.
- Tapping recurrence or frequency.
- Effective start and end dates.
- Planned number of trees.
- Schedule status.
- Creation and modification audit fields.

Schedule statuses:

```
DRAFT
ACTIVE
SUSPENDED
CLOSED
```

Schedule generation must be idempotent and must never duplicate existing assignments.

FR-02: Tapping Assignment Management

Support:

- Creating assignments from schedules.
- Explicit ad hoc assignments when authorized.
- Assigning a worker.
- Identifying estate, block, and tapping date.
- Recording planned tree count.
- Starting work.
- Completing work.
- Recording missed work.
- Cancelling assignments.
- Filtering by estate, block, worker, date, and status.

Assignment statuses:

```
PLANNED
IN_PROGRESS
COMPLETED
MISSED
CANCELLED
```

Enforce valid state transitions.

FR-03: Tapping Execution

Record:

- Assignment ID.
- Worker ID.
- Estate ID.
- Block ID.
- Tapping date.
- Planned tree count.
- Actual trees tapped.
- Work start and completion timestamps.
- Optional operational remarks.
- Recording actor and audit timestamps.

Do not capture latex weight or DRC in this milestone.

FR-04: Missed Tapping

Support missed-reason codes:

```
RAIN
WORKER_UNAVAILABLE
TREE_RESTING
ACCESS_RESTRICTED
OTHER
```

A missed assignment requires a valid reason and an authenticated recording actor.

Prevent completion of cancelled or missed assignments unless an explicit reopening workflow is subsequently designed.

FR-05: Operational Productivity

Expose basic operational summaries:

- Planned tapping assignments.
- Completed assignments.
- Missed assignments.
- Completion percentage.

- Planned trees.
- Actually tapped trees.
- Worker-level productivity.
- Block-level tapping progress.

Define calculation semantics, date boundaries, and zero-denominator behavior.

Do not implement an analytics data warehouse or AI forecasting in this milestone.

5. Domain Model

Use Domain-Driven Design with clear separation of:

```
api/
application/
domain/
infrastructure/
    persistence/
    messaging/
    integration/
config/
```

Adapt to the repository's actual package conventions.

Implement these domain concepts:

TappingSchedule

```
id: UUID
estateId: UUID
blockId: UUID
name: String
recurrenceRule: String
plannedTreeCount: Integer
effectiveFrom: LocalDate
effectiveTo: LocalDate?
status: ScheduleStatus
```

```
createdAt: OffsetDateTime
updatedAt: OffsetDateTime
version: Long
```

TappingAssignment

```
id: UUID
scheduleId: UUID?
estateId: UUID
blockId: UUID
workerId: String
tappingDate: LocalDate
plannedTreeCount: Integer
status: AssignmentStatus
createdAt: OffsetDateTime
updatedAt: OffsetDateTime
version: Long
```

TappingRecord

```
id: UUID
assignmentId: UUID
workerId: String
estateId: UUID
blockId: UUID
tappingDate: LocalDate
actualTreeCount: Integer
startedAt: OffsetDateTime
completedAt: OffsetDateTime?
notes: String?
createdBy: String
createdAt: OffsetDateTime
updatedAt: OffsetDateTime
```

TappingMissedReason

```
assignmentId: UUID
reasonCode: MissedReasonCode
remarks: String?
recordedAt: OffsetDateTime
recordedBy: String
```

Prefer aggregates and domain services that enforce business invariants without coupling controllers directly to JPA entities.

6. Business Rules

Implement and test these rules:

1. Planned tree count must be positive.
2. Actual tree count must be nonnegative.
3. By default, actual tree count cannot exceed planned tree count.
4. Do not allow duplicate active assignments for the same worker, block, and tapping date.
5. Only active schedules may generate assignments.
6. Assignments cannot be completed without a valid execution record.
7. Completed assignments cannot be silently changed.
8. Missed assignments must include a reason.
9. Assignment completion and missed marking must be safe under retries.
10. Concurrent modifications must be protected with optimistic locking or equivalent database safeguards.
11. Schedule generation must prevent duplicate work occurrences.
12. Future assignments must not automatically be marked missed.
13. Use estate-local business dates, with explicit time-zone semantics.
14. Schedule changes must not retroactively modify completed work.

Document the recurrence, cancellation, date-handling, and idempotency policies.

7. PostgreSQL and Flyway

Create or extend the operations-service database schema using Flyway.

Suggested tables:

```
tapping_schedule
tapping_assignment
tapping_record
tapping_missed_reason
outbox_event
```

Use:

- UUID primary keys.
- Referential integrity within the operations database.
- Appropriate uniqueness constraints.
- CHECK constraints for numeric values.
- Optimistic-lock version columns.
- Indexes for common filters.
- UTC audit timestamps.

Do not add database-level relationships to estate-service or spatial-service databases.

Write integration tests that apply migrations against real PostgreSQL through Testcontainers.

8. REST API Contracts

Follow existing API naming conventions where confirmed.

Proposed endpoints:

```
POST   /api/v1/tapping/schedules
GET    /api/v1/tapping/schedules
GET    /api/v1/tapping/schedules/{id}
PATCH  /api/v1/tapping/schedules/{id}
POST   /api/v1/tapping/schedules/{id}/activate
POST   /api/v1/tapping/schedules/{id}/suspend
POST   /api/v1/tapping/schedules/{id}/close
POST   /api/v1/tapping/schedules/{id}/generate-assignments
```

```
POST   /api/v1/tapping/assignments
GET    /api/v1/tapping/assignments
GET    /api/v1/tapping/assignments/{id}
POST   /api/v1/tapping/assignments/{id}/start
POST   /api/v1/tapping/assignments/{id}/complete
POST   /api/v1/tapping/assignments/{id}/mark-missed
POST   /api/v1/tapping/assignments/{id}/cancel

GET    /api/v1/tapping/records
GET    /api/v1/tapping/summary
```

Requirements:

- DTO-based requests and responses.
- Bean Validation.
- Pagination and filtering.
- Appropriate HTTP status codes.
- Standardized problem responses.
- Request correlation IDs.
- Documented idempotency behavior.
- Versioned OpenAPI specifications.
- Compatible gateway routing.

Implement end-to-end API tests, including failure cases.

9. RabbitMQ Domain Events

Publish versioned events for meaningful state transitions:

```
TappingScheduleActivated.v1
TappingAssignmentCreated.v1
TappingAssignmentStarted.v1
TappingCompleted.v1
TappingMissed.v1
TappingAssignmentCancelled.v1
```

Follow the existing exchange and routing-key conventions.

Suggested envelope:

```
{
  "eventId": "UUID",
  "eventType": "TappingCompleted",
  "schemaVersion": 1,
  "occurredAt": "ISO-8601 UTC timestamp",
  "producer": "agrosphere-operations-service",
  "correlationId": "UUID",
  "aggregateId": "UUID",
  "aggregateVersion": 2,
  "payload": {}
}
```

Provide event JSON schemas and example payloads in `agrosphere-contracts`.

Mandatory Transactional Outbox

Implement the transactional outbox pattern:

1. Update operational data in a database transaction.
2. Write the associated event to the outbox in the same transaction.
3. Publish asynchronously to RabbitMQ.
4. Use publisher confirms to determine successful broker publication.
5. Retry failed publications with bounded backoff.
6. Preserve a stable event ID across retries.
7. Mark events as published only after confirmation.

Document at-least-once delivery semantics and downstream consumer deduplication expectations.

Do not publish domain events before database commit.

10. Estate-Service Integration

Verify the real estate-service APIs.

Use published APIs to validate estate and block identifiers.

Do not access the estate-service database directly.

If a required validation endpoint is missing, add the smallest safe endpoint in the estate-service repository with corresponding tests and contracts, or report an explicit blocker.

Include:

- Timeouts.
- Service authentication.
- Error mapping.
- Unavailable-dependency behavior.
- Estate-scoped authorization.

Do not invent success responses when estate validation fails.

11. Security

Reuse existing Keycloak and Spring Security conventions.

Suggested roles:

```
PLANTATION_ADMIN
ESTATE_MANAGER
FIELD_SUPERVISOR
FIELD_WORKER
AUDITOR
```

Map these to verified existing claims, roles, or scopes.

Authorization rules:

- Admins manage all tapping workflows.
- Estate managers manage their authorized estates.
- Supervisors manage authorized field assignments.
- Workers can view and update authorized assignments belonging to them.
- Auditors have read-only permissions.

Enforce object-level and estate-level authorization inside the service, not only at the API Gateway.

Never trust arbitrary client-submitted worker IDs as authentication proof.

Secure service-to-service calls.

Do not commit secrets.

12. Automated Testing

Implement comprehensive automated tests.

Unit tests

- Schedule creation and lifecycle.
- Recurrence expansion.
- Assignment generation.
- Domain invariants.
- Valid and invalid state transitions.
- Tapping completion.
- Missed-tapping reasons.
- Productivity calculations.
- Date and time-zone boundaries.

Integration tests

- PostgreSQL/Flyway migrations.
- Database constraints.
- REST APIs with JWT authentication.
- Role and estate authorization.
- Optimistic-lock conflicts.
- Duplicate-request handling.
- Outbox persistence.
- RabbitMQ publication and retries.

Contract tests

- OpenAPI contract alignment.
- Event JSON schema validation.
- Backward-compatible event versioning rules.

Use Testcontainers for PostgreSQL and RabbitMQ where appropriate.

Do not claim integration tests passed if Docker or the containers were unavailable.

13. Nonfunctional Requirements

Implement:

- Independently deployable operations service.
- Structured logging and correlation IDs.
- Spring Boot Actuator health/readiness endpoints.
- Outbox backlog and failure metrics.
- Graceful shutdown.
- Appropriate database connection pooling.
- Pagination for collection queries.
- Database indexing for operational filters.
- Safe retry and failure behavior.
- Configurable timeouts and environment-specific configuration.
- UTC timestamp storage and documented estate-local business-date handling.
- No sensitive information in logs.

Avoid premature distributed caching or additional infrastructure not required by this milestone.

14. Docker Compose and CI/CD

Inspect and extend existing infrastructure configuration.

Ensure the operations service can run locally with:

- Its own PostgreSQL database.
- RabbitMQ.
- Keycloak authentication.
- API Gateway routing.

- Existing estate service where required.

Update service-specific CI to execute:

1. Compilation.
2. Unit tests.
3. Integration tests.
4. Contract validation.
5. Application packaging.

Use existing CI conventions rather than introducing a competing system.

15. Architecture Documentation and Diagrams

Update the `agrosphere-architecture` repository.

Create PlantUML source diagrams for:

1. C4 container relationships.
2. Operations-service component architecture.
3. Tapping domain/class diagram.
4. Tapping assignment state machine.
5. Schedule-to-assignment generation sequence.
6. Completion-to-outbox-to-RabbitMQ sequence.
7. Database ER diagram.

Also provide:

- API usage examples.
- Event contract descriptions.
- Security role matrix.
- Architecture Decision Records.
- Local development guide.
- Failure and recovery scenarios.

Diagrams must match the implemented solution.

16. Definition of Done

Milestone 3A is complete only when all of the following have been verified:

- [ ] Existing repository capabilities have been inspected and documented.
- [ ] Tapping schedules can be created and managed.
- [ ] Scheduled assignments can be generated without duplication.
- [ ] Workers and supervisors can perform authorized tapping workflows.
- [ ] Execution records persist correctly.
- [ ] Missed tapping requires a valid reason.
- [ ] Summary APIs calculate accurate operational metrics.
- [ ] Flyway migrations run successfully.
- [ ] RabbitMQ events are published using the transactional outbox.
- [ ] OpenAPI and event contracts reflect the implementation.
- [ ] Estate-scoped authorization is enforced.
- [ ] Unit tests pass.
- [ ] PostgreSQL and RabbitMQ integration tests pass.
- [ ] Docker local integration has been verified.
- [ ] Independent service build succeeds.
- [ ] CI configuration is updated and validated.
- [ ] PlantUML architecture diagrams are updated.
- [ ] No direct cross-service database access has been introduced.

Mark any item that could not be verified as **NOT VERIFIED** rather than complete.

17. Completion Report

Create:

`agrosphere-architecture/docs/milestones/milestone-3a-completion.md`

Include:

- Executive summary.
- Repository changes.

Take your curiosity further
Scan to get the ChatGPT app

- Domain entities implemented.
- Database migration details.
- REST endpoint inventory.
- Event contract inventory.
- Security implementation.
- Tests run and their actual results.
- Docker integration results.
- Architectural decisions.
- Known issues and limitations.
- Unresolved prerequisites.
- Recommendations for Milestone 3B.

Conclude your Codex response with:

```
AGROSPHERE MILESTONE 3A COMPLETION REPORT

Status: COMPLETE / PARTIAL / BLOCKED

Repositories Changed:
...

Features Implemented:
...

REST APIs:
...

Domain Events:
...

Database Migrations:
...

Automated Tests:
...
```

Take your curiosity further
Scan to get the ChatGPT app

```
Docker Integration:
...

Documentation:
...

Outstanding Issues:
...

Readiness for Milestone 3B:
READY / NOT READY
```

18. Execution Instructions

Execute in this order:

Phase 1 — Discovery: Inspect repositories, contracts, and implementation prerequisites.

Phase 2 — Domain Design: Finalize entities, aggregates, state transitions, invariants, and event schemas.

Phase 3 — Backend Implementation: Implement database migrations, domain logic, services, repositories, and REST controllers.

Phase 4 — Integration: Implement estate-service validation, Keycloak security, RabbitMQ events, and the transactional outbox.

Phase 5 — Quality: Implement and execute automated tests, address defects, and verify builds.

Phase 6 — Documentation: Update PlantUML diagrams, contracts, ADRs, and local-development instructions.

Phase 7 — Completion: Produce a detailed completion report with verified results.

Proceed autonomously through all phases. Do not stop after generating a plan or architecture document.

Do not automatically start Milestone 3B.

Take your curiosity further
Scan to get the ChatGPT app

Once Codex finishes, its completion report will be the basis for **Milestone 3B — Latex Collection, Weighment, DRC, and Production Reconciliation**.

I recommend keeping 3B separate because the tapping records created in 3A will become its operational foundation.

You

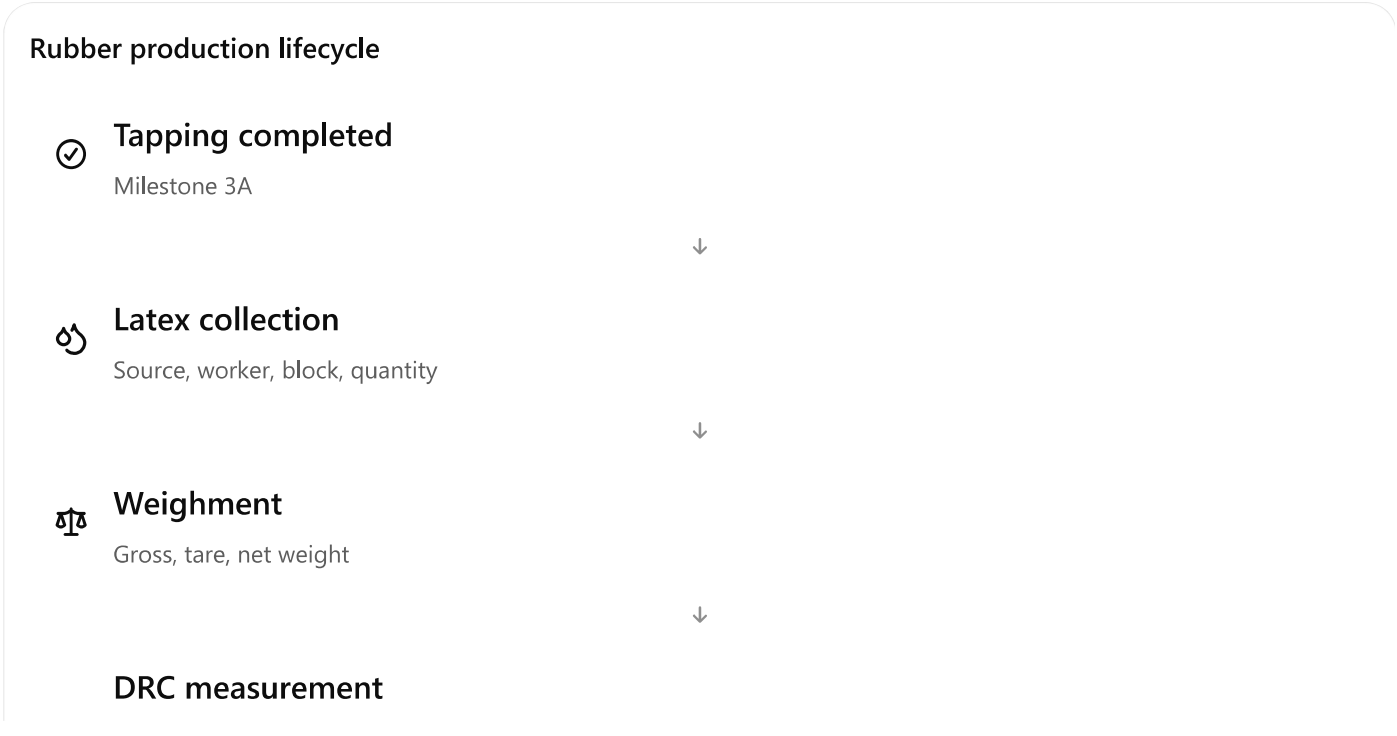
Please generate the prompt for Milestone 3B.

ChatGPT

AgroSphere — Milestone 3B: Latex Collection, Weighment, DRC & Production Reconciliation

Milestone 3B extends the Rubber Tapping Operations implemented in 3A into a complete **rubber production tracking workflow**.

The key objective is to establish end-to-end traceability:



Take your curiosity further
Scan to get the ChatGPT app

Test results and dry-rubber content

↓

Production reconciliation

Collections, measurements, variances

↓

Operational events

RabbitMQ → future analytics

Primary implementation repository: `agrosphere-operations-service`

Supporting repositories: `agrosphere-contracts`, `agrosphere-infrastructure`, `agrosphere-api-gateway`, and `agrosphere-architecture`.

No new microservice is necessary for 3B. Latex collections, weighments, and production reconciliation belong to the existing operations bounded context, with separate internal modules.

The following Codex prompt is designed to implement 3B end to end, including contracts, migrations, security, RabbitMQ, automated testing, documentation, and a verifiable completion report.

Codex implementation prompt — Milestone 3B

Copy the complete prompt below into Codex in your existing AgroSphere multi-repository VS Code workspace.

Writing

AgroSphere — Milestone 3B: Latex Collection, Weighment, Dry

Rubber Content (DRC) and Production Reconciliation

Role and Objective

You are the **Lead Solution Architect, Senior Java Engineer, Domain-Driven Design Specialist, and Integration Engineer** for AgroSphere, an AI-powered Rubber Plantation Management and Digital Twin platform.

Implement **Milestone 3B — Latex Collection, Weighment, DRC and Production Reconciliation** in the existing AgroSphere multi-repository architecture.

Milestone 3A introduced rubber tapping schedules, worker assignments, tapping execution records, and domain events.

Milestone 3B must extend that operational foundation into an auditable rubber-production workflow.

Primary business workflow:

1. Workers complete rubber tapping activities.
2. Collected latex is registered against its source estate/block and, where available, tapping assignments.
3. Latex is delivered to a collection point.
4. Weighment captures gross, tare, and net weights.
5. Authorized personnel record Dry Rubber Content measurements.
6. The system calculates estimated dry-rubber quantity.
7. Supervisors reconcile collected, accepted, rejected, and transferred quantities.
8. The system publishes domain events for future analytics and digital twin visualization.

Implement working production-quality functionality, not just scaffolding or architectural documents.

---

1. Mandatory Engineering Instructions

1. Inspect all relevant repositories before modifying code.
2. Verify the actual Milestone 3A implementation and completion report.
3. Do not assume 3A endpoints, entities, services, events, or infrastructure exist without verification.

4. Preserve strict microservices and multi-repository boundaries.
5. No cross-service database access.
6. Each service must remain independently deployable, testable, and buildable.
7. Reuse established coding conventions and technologies.
8. Use Java 21 and the existing compatible Spring Boot version.
9. Use PostgreSQL and the established Flyway migration strategy.
10. Preserve Keycloak/OIDC security and API Gateway integration.
11. Reuse RabbitMQ and transactional-outbox infrastructure from 3A when verified.
12. Do not duplicate estate, block, employee, or spatial master-data ownership.
13. Implement idempotency and transactional integrity.
14. Do not silently overwrite confirmed production measurements.
15. Use accurate decimal arithmetic for all weight, quantity, and DRC calculations.
16. Do not ask unnecessary implementation questions. Choose sensible defaults and document them.
17. Do not introduce Milestone 3C, 3D, 3E, or 3F functionality.
18. Do not push commits or deploy to shared environments without authorization.
19. Preserve unrelated working changes.
20. Report only builds, tests, and integrations that actually executed.

If a required feature is missing from Milestone 3A, implement the smallest safe, backward-compatible extension or explicitly report the blocker.

---

2. Repository Discovery and Prerequisite Verification

Inspect:

```
agrosphere-architecture
agrosphere-contracts
agrosphere-infrastructure
agrosphere-api-gateway
agrosphere-estate-service
agrosphere-spatial-service
agrosphere-operations-service
agrosphere-web
```

Additional repositories may be inspected if necessary, but avoid unrelated modifications.

Verify:

- Existing tapping assignments and execution records.
- Tapping completion lifecycle.
- Existing estate/block identifiers.
- Tapping-related REST APIs.
- RabbitMQ event conventions.
- Transactional outbox behavior.
- Database migrations.
- Keycloak roles and JWT mappings.
- Existing test frameworks.
- Docker Compose integration.
- OpenAPI specification conventions.
- CI/CD workflows.

Determine whether the operations service already has production, collection, inventory, or measurement models.

Reuse compatible existing functionality rather than introducing duplicates.

Create:

`agrosphere-architecture/docs/milestones/milestone-3b-discovery.md`

Include a clear dependency-verification matrix:

Dependency	Expected Capability	Verified	Action
Milestone 3A	Tapping records	Yes/No	Reuse/Fix/Block
Estate service	Estate/block validation	Yes/No	Reuse/Fix/Block
RabbitMQ	Event exchange	Yes/No	Reuse/Fix/Block

Outbox	Transactional publication	Yes/No	Reuse/Fix/Block
Keycloak	Auth and roles	Yes/No	Reuse/Fix/Block
Infrastructure	PostgreSQL	Yes/No	Reuse/Fix/Block

Do not stop after this document. Continue with implementation unless an actual blocker prevents safe progress.

---

3. Architecture and Ownership

Primary repository

`agrosphere-operations-service`

Create separate internal modules within the existing microservice, following actual package conventions.

Suggested structure:

```
operations/
  tapping/
  latexcollection/
    api/
    application/
    domain/
    infrastructure/
  weighment/
    api/
    application/
    domain/
    infrastructure/
  drc/
    api/
    application/
```

```
    domain/
    infrastructure/
  reconciliation/
    api/
    application/
    domain/
    infrastructure/
  shared/
```

These are **internal domain modules**, not independently deployed microservices.

Avoid unnecessary shared mutable domain models.

Supporting repositories

agrosphere-contracts

- OpenAPI specifications.
- Request/response schemas.
- RabbitMQ event contracts.
- JSON Schema definitions.
- Sample payloads.
- Contract compatibility checks.

agrosphere-infrastructure

- Docker Compose integration.
- PostgreSQL configuration.
- RabbitMQ settings.
- Test support.
- Environment variable documentation.

agrosphere-api-gateway

- Expose the new operations endpoints through existing routing conventions.
- Preserve authentication and gateway security.

agrosphere-architecture

- PlantUML diagrams.
- Domain documentation.
- ADRs.
- API and event documentation.
- Completion report.

Database ownership

All collection, weighment, DRC, reconciliation, and related outbox data belong to the operations-service database.

Never directly query the estate-service, spatial-service, or analytics-service databases.

---

4. Functional Requirements

FR-01: Latex Collection Registration

Implement registration of latex collection from plantation tapping activities.

Each collection should identify:

- Collection ID.
- Estate ID.
- Block ID.
- Source tapping assignment/record, where applicable.
- Collection point identifier.
- Collection date and time.
- Worker or collection team identifier.
- Collection container identifier, where applicable.
- Collection type.
- Reported quantity, where applicable.
- Collection status.
- Remarks.

- Audit information.

Supported collection types:

```
FIELD_LATEX
CUP_LUMP
SCRAP
OTHER
```

Important domain distinction: Dry Rubber Content is primarily relevant to fresh or liquid latex. Cup lump and scrap must not automatically be processed using liquid-latex DRC assumptions.

Use explicit collection-type policies.

Do not imply all rubber products use an identical calculation method.

Collection lifecycle

Suggested states:

```
REGISTERED
RECEIVED
WEIGHED
ACCEPTED
REJECTED
CANCELLED
```

Use domain-appropriate state transitions.

A collection may reference an existing tapping record, but the design must also allow a controlled exception for collections without a matching 3A record, subject to authorization and audit requirements.

Support multiple collections from a tapping assignment when legitimate.

Prevent duplicate registration using stable collection identifiers/idempotency keys rather than assuming a one-to-one assignment relationship.

Validate estate/block relationships using existing authoritative contracts.

FR-02: Latex Collection Batch Management

Implement collection batches that group one or more compatible source collections.

Batch information:

```
batchId
batchNumber
estateId
collectionPointId
collectionDate
collectionType
status
createdAt
createdBy
```

Support:

- Creating batches.
- Adding collections.
- Removing unconfirmed collection associations.
- Viewing batch details.
- Tracking source provenance.
- Confirming a batch.
- Rejecting a batch.
- Closing a batch.

Define explicit rules for mixing collection types.

Default: do not mix FIELD_LATEX with CUP_LUMP or SCRAP within the same batch.

Preserve relationships between every source collection and its batch.

Do not permit a source quantity to be double-counted across multiple active batches.

When partial allocations are required, represent allocated quantities explicitly and enforce that their sum cannot exceed the source's available measured quantity.

If partial allocation is unnecessary for initial scope, enforce one active batch membership per collection and document the limitation.

Batch confirmation must be transactional and idempotent.

FR-03: Latex Weighment

Implement weighment capture.

Fields:

```
weighmentId: UUID
batchId: UUID
measurementTime: OffsetDateTime
grossWeightKg: BigDecimal
tareWeightKg: BigDecimal
netWeightKg: BigDecimal
weighingDeviceId: String?
measuredBy: String
remarks: String?
status: WeighmentStatus
```

Calculation:

Net Weight (kg) = Gross Weight (kg) − Tare Weight (kg)

Business rules:

- Gross weight must be positive.
- Tare weight must be nonnegative.
- Tare cannot exceed gross.
- Net weight must be positive for an accepted material batch.
- Use BigDecimal, never float or double.
- Store consistent measurement units.

- Configurable weight scale and rounding policy.
- Prevent duplicate confirmed weighments.
- Permit controlled reweighment with documented supersession.
- Preserve previous measurement history.
- Restrict confirmation/correction to authorized users.
- Record device, actor, timestamp, and correction reason.

Suggested statuses:

```
DRAFT
CONFIRMED
SUPERSEDED
VOIDED
```

A confirmed measurement must be immutable.

Corrections must create new records and preserve historical measurements.

Weighment and source allocation

Where multiple collections contribute to one batch, store their provenance and distinguish batch-level measured weight from source-level estimated quantities.

Do not claim precise source-level measured dry-rubber yield unless the allocation method is explicitly defined and documented.

FR-04: Dry Rubber Content (DRC) Measurement

Implement recording and validation of DRC measurements.

Domain model:

```
drcMeasurementId: UUID
batchId: UUID
weighmentId: UUID
drcPercentage: BigDecimal
```

```
measurementMethod: String
sampleId: String?
sampledAt: OffsetDateTime?
testedAt: OffsetDateTime
testedBy: String
status: DrcMeasurementStatus
remarks: String?
```

Supported method:

```
LAB_TEST
```

Additional methods may be modeled only if requirements and calculation semantics are explicitly documented.

Do not fabricate laboratory measurements from arbitrary sensor data.

DRC calculation

For eligible liquid latex:

Estimated Dry Rubber Weight (kg) = Net Latex Weight (kg) × DRC (%) / 100

Example:

```
Net Latex Weight: 250.000 kg
DRC: 32.50%
Estimated Dry Rubber: 81.250 kg
```

Use BigDecimal and centralized calculation services.

Business rules:

- DRC percentage must be greater than zero and at most 100.
- DRC must be associated with the applicable batch/weighment.
- Only confirmed weighments may be used for an accepted production calculation.
- Retesting requires an explicit new measurement record.

- Confirmed DRC values cannot be edited in place.
- Define which confirmed measurement is authoritative when retests exist.
- Store measurement method and audit metadata.
- Define precision/rounding in a documented policy.
- Changing an authoritative weighment or DRC result must invalidate or require recalculation of affected draft production results.

Include a domain policy explaining how non-liquid rubber types are handled. Do not automatically invent DRC values or claim dry-rubber results for unsupported methods.

Use a documented example test:

```
Net latex = 250.000 kg
DRC = 32.50%
Expected dry rubber = 81.250 kg
```

FR-05: Production Quantity and Yield Records

Implement production records for confirmed batch measurements.

Suggested model:

```
productionRecordId
batchId
estateId
collectionPointId
productionDate
productType
netWeightKg
drcPercentage?
dryRubberWeightKg?
sourceWeighmentId
sourceDrcMeasurementId?
status
createdAt
```

```
confirmedAt?
createdBy
```

Supported states:

```
DRAFT
CONFIRMED
VOIDED
```

Rules:

- Every confirmed production record must reference the source measurement evidence required by its product-type policy.
- Avoid duplication of confirmed production records for one authoritative batch measurement set.
- Preserve original measurements and calculation versions.
- Do not overwrite confirmed quantities.
- Corrections must follow controlled reversal/replacement.
- Clearly distinguish physical measured weight from calculated dry-rubber equivalent.
- Record calculation policy version and rounding convention.
- Only publish final production confirmation events when the production record is valid and confirmed.

Calculate block-level production only where source allocation is traceable and defensible.

Do not invent equal allocation of mixed batches across blocks.

---

5. Production Reconciliation

FR-06: Reconciliation Workflow

Implement reconciliation between registered collections, received material, confirmed weighments, and production records.

Suggested reconciliation entity:

```
reconciliationId
batchId
reportedCollectionWeightKg?
confirmedNetWeightKg
varianceKg?
variancePercentage?
status
remarks
reviewedBy
reviewedAt
createdAt
updatedAt
```

If reported quantity and confirmed net weight are comparable:

```
Variance Kg = Confirmed Net Weight − Reported Collection Weight

Variance % = Variance Kg / Reported Collection Weight × 100
```

Handle zero or missing denominators explicitly.

Never silently compare different units, estimated volume, or other non-comparable measurements.

Suggested statuses:

```
PENDING
MATCHED
VARIANCE_REVIEW
APPROVED
REJECTED
```

Requirements:

1. Calculate variance deterministically.
2. Define configurable tolerance thresholds.

3. Automatically flag out-of-tolerance batches for review.
4. Require supervisor approval where appropriate.
5. Record variance reasons and resolutions.
6. Preserve reconciliation history.
7. Do not allow unreviewed out-of-tolerance batches to be finalized.
8. Do not reconcile missing/invalid measurement data as zero.
9. Prevent double counting across cancelled, rejected, or superseded batches.
10. Preserve the provenance of final quantities.

Distinguish operational reconciliation from physical inventory control. Do not build a full inventory, accounting, procurement, or transportation system in this milestone.

---

6. REST API Requirements

Follow verified existing naming and API versioning conventions.

Proposed APIs:

Latex collections

```
POST   /api/v1/latex/collections
GET    /api/v1/latex/collections
GET    /api/v1/latex/collections/{id}
POST   /api/v1/latex/collections/{id}/receive
POST   /api/v1/latex/collections/{id}/cancel
```

Collection batches

```
POST   /api/v1/latex/batches
GET    /api/v1/latex/batches
GET    /api/v1/latex/batches/{id}
POST   /api/v1/latex/batches/{id}/collections
DELETE /api/v1/latex/batches/{id}/collections/{collectionId}
POST   /api/v1/latex/batches/{id}/confirm
POST   /api/v1/latex/batches/{id}/close
```

Weighment

```
POST   /api/v1/latex/batches/{id}/weighments
GET    /api/v1/latex/batches/{id}/weighments
POST   /api/v1/latex/weighments/{id}/confirm
POST   /api/v1/latex/weighments/{id}/supersede
```

DRC

```
POST   /api/v1/latex/batches/{id}/drc-measurements
GET    /api/v1/latex/batches/{id}/drc-measurements
POST   /api/v1/latex/drc-measurements/{id}/confirm
```

Production

```
POST   /api/v1/latex/batches/{id}/production-records
GET    /api/v1/latex/production-records
GET    /api/v1/latex/production-records/{id}
POST   /api/v1/latex/production-records/{id}/confirm
```

Reconciliation

```
POST   /api/v1/latex/batches/{id}/reconciliations
GET    /api/v1/latex/reconciliations
GET    /api/v1/latex/reconciliations/{id}
POST   /api/v1/latex/reconciliations/{id}/review
POST   /api/v1/latex/reconciliations/{id}/approve
POST   /api/v1/latex/reconciliations/{id}/reject
```

Adjust routes where existing conventions require it.

All mutating APIs must enforce authorization and optimistic concurrency or equivalent transactional controls.

Implement:

- Bean Validation.
- Standardized error responses.
- Pagination and filtering.
- Date-range filters.
- Estate/block filters.
- Batch status filters.
- Correlation IDs.
- Idempotency for retriable commands.
- Correct 400, 401, 403, 404, 409, and 422 behavior as appropriate.
- Accurate and versioned OpenAPI schemas.
- API Gateway routing.

Provide usable request/response examples for the primary workflow.

---

7. Database Design

Use the existing operations-service PostgreSQL database.

Suggested tables:

```
latex_collection
latex_batch
latex_batch_collection
latex_weighment
latex_drc_measurement
rubber_production_record
latex_reconciliation
latex_reconciliation_review
```

Reuse existing outbox infrastructure.

Use Flyway migrations.

Requirements:

- UUID primary keys.
- Foreign keys within the operations-service database only.
- Relevant uniqueness constraints.
- Weight and DRC numeric precision.
- CHECK constraints for valid quantities.
- Database-enforced prevention of duplicate confirmed production.
- Status and audit timestamps.
- Optimistic-lock version columns.
- Indexes supporting filters and joins.
- Clear correction/supersession relationships.
- Source traceability.
- Safe additive migrations.

Use a documented NUMERIC precision/scale, with BigDecimal mappings and one centralized rounding policy.

Do not introduce distributed database transactions.

Ensure a batch's confirmed weight, DRC, production, and reconciliation states cannot become inconsistent under concurrent updates.

---

8. Domain Events and RabbitMQ

Reuse the transactional outbox architecture verified in Milestone 3A.

Introduce versioned events as appropriate:

```
LatexCollectionRegistered.v1
LatexCollectionReceived.v1
LatexBatchConfirmed.v1
LatexWeighmentConfirmed.v1
LatexDrcMeasured.v1
LatexProductionConfirmed.v1
LatexReconciliationApproved.v1
LatexBatchRejected.v1
```

Do not publish unsupported or semantically premature event types.

For example, distinguish a recorded draft DRC reading from an authoritative confirmed measurement.

All events must use the existing standard envelope or a backward-compatible extension.

Include:

```
eventId
eventType
schemaVersion
occurredAt
producer
correlationId
aggregateId
aggregateVersion
payload
```

Useful payload fields may include:

- Estate ID.
- Block IDs, only when traceable.
- Collection/batch ID.
- Product type.
- Net weight.
- DRC percentage, if applicable.
- Dry-rubber weight, if calculated.
- Production date.
- Reconciliation status.

Never publish misleading block-level yield allocations.

Requirements:

- Transactional outbox publication.
- Stable event ID.

- Publisher confirmations.
- Retry handling.
- At-least-once delivery.
- Idempotent downstream processing expectations.
- Versioned JSON schemas.
- Contract validation tests.
- Documented exchange and routing keys.

Do not implement the future analytics consumers in this milestone.

---

9. Security and Authorization

Reuse actual Keycloak security settings and verified role mappings.

Suggested roles:

```
PLANTATION_ADMIN
ESTATE_MANAGER
FIELD_SUPERVISOR
FIELD_WORKER
COLLECTION_OPERATOR
WEIGHBRIDGE_OPERATOR
LAB_TECHNICIAN
AUDITOR
```

Map new roles to existing configuration where possible.

Suggested permissions:

Operation	Authorized Role
Register collection	Field worker / Supervisor
Receive collection	Collection operator / Supervisor

Create batch	Collection operator
Record weighment	Weighbridge operator
Confirm weighment	Authorized weighbridge operator / Supervisor
Enter DRC result	Lab technician
Confirm DRC	Authorized lab technician
Approve reconciliation	Estate manager / Authorized supervisor
View reports	Authorized operational users / Auditor
Correct confirmed records	Privileged authorized roles with audit trail

Enforce estate-scoped access inside the service.

Derive user identity from verified JWT claims.

Never trust client-submitted actor identities.

Ensure corrections, reversals, and approvals leave a complete audit trail.

Do not weaken existing security to simplify local integration.

10. Automated Testing

Implement comprehensive tests.

Unit tests

Cover:

- Collection registration validation.
- Batch grouping rules.

- Duplicate registration handling.
- Collection-to-batch allocation constraints.
- Weighment calculation.
- Invalid tare/gross quantities.
- DRC calculation.
- DRC boundary conditions.
- Rounding and precision.
- Product-type policy.
- Production confirmation.
- Reconciliation tolerances.
- Zero-denominator variance handling.
- Invalid lifecycle transitions.
- Measurement supersession.
- Duplicate confirmation.
- Concurrent workflow conflicts.

Include these numerical examples:

Example 1 — Weighment

```
Gross = 285.500 kg
Tare = 35.500 kg
Net = 250.000 kg
```

Expected net weight = `250.000 kg`.

Example 2 — DRC

```
Net = 250.000 kg
DRC = 32.50%
Dry rubber = 81.250 kg
```

Expected = `81.250 kg`.

Example 3 — Reconciliation

```
Reported = 255.000 kg
Confirmed = 250.000 kg

Variance = -5.000 kg
Variance % ≈ -1.9608%
```

Assert the policy-defined display rounding and underlying calculation precision separately.

Integration tests

Use Testcontainers where appropriate for:

- PostgreSQL.
- Flyway.
- RabbitMQ.
- Outbox delivery.
- REST endpoints.
- JWT authorization.
- Estate-scoped access.
- Transactional workflows.
- Database uniqueness.
- Concurrent updates.

Contract tests

Validate:

- OpenAPI specifications.
- JSON Schema event contracts.
- Representative real event payloads.
- API backward compatibility.

Use realistic test fixtures.

Do not mark tests successful unless executed.

11. Observability, Resilience and Nonfunctional Requirements

Implement or reuse:

- Structured logs.
- Correlation IDs.
- Spring Boot Actuator.
- Readiness and health probes.
- Database and broker health checks.
- Metrics for collections, weighments, DRC confirmations, rejected batches, and reconciliation exceptions.
- Outbox backlog and failure monitoring.
- Bounded service-to-service timeouts.
- Retry policies.
- Graceful shutdown.
- Pagination.
- Configurable measurement precision/tolerances.
- Safe handling of duplicate submissions.
- Explicit handling of unavailable estate-service dependencies.

Avoid unbounded retries or swallowing errors.

Do not log confidential tokens or unnecessary personal data.

12. Docker and CI/CD Integration

Inspect existing Docker Compose and CI workflows.

Update `agrosphere-infrastructure` only as necessary to integrate the enhanced operations service.

Ensure local integration supports:

- Operations service.
- Operations PostgreSQL.
- RabbitMQ.
- Keycloak.
- API Gateway.
- Estate service when required.

Do not introduce extra databases or messaging brokers without need.

Ensure that the operations repository remains independently buildable.

CI requirements:

1. Compile.
2. Execute unit tests.
3. Execute integration tests.
4. Validate contracts.
5. Package the application.
6. Perform compatible automated quality checks already used by the project.

Report any unavailable CI runtime validation separately.

---

13. Architecture Documentation

Update `agrosphere-architecture`.

Create PlantUML source files for:

1. C4 container view showing operations and related services.
2. Component diagram for collection, weighment, DRC, and reconciliation.
3. Production domain model/class diagram.
4. Collection and batch state machines.
5. Latex collection-to-production sequence diagram.
6. Transactional outbox publication sequence.
7. Database entity-relationship diagram.

8. Exception handling and reconciliation sequence.

Document:

- Business process flow.
- Domain invariants.
- Measurement units and precision.
- DRC calculation policy.
- Supported product types.
- Batch provenance and allocation strategy.
- Correction and supersession workflow.
- Reconciliation rules.
- Security permissions.
- API examples.
- Event contracts.
- ADRs.
- Future integration points with analytics and digital twin services.

Diagrams must represent verified code.

Do not invent integrations that have not been implemented.

---

14. Definition of Done

Milestone 3B is complete only when:

- [ ] Milestone 3A integration has been verified.
- [ ] Collection registration works.
- [ ] Collections can be associated with batches.
- [ ] Batch quantities cannot be double-counted.
- [ ] Weighment calculates accurate net weight.
- [ ] Measurement correction history is preserved.
- [ ] DRC supports validated measurements.
- [ ] Dry-rubber calculation uses BigDecimal correctly.

- [ ] Unsupported product types are not assigned fabricated DRC.
- [ ] Production records preserve source provenance.
- [ ] Production confirmation is idempotent.
- [ ] Reconciliation and exception review work.
- [ ] Out-of-tolerance reconciliation requires approval.
- [ ] REST APIs are implemented and documented.
- [ ] Keycloak authentication and role/estate authorization are enforced.
- [ ] RabbitMQ events use versioned schemas and the transactional outbox.
- [ ] Flyway migrations execute successfully.
- [ ] Unit tests pass.
- [ ] PostgreSQL/RabbitMQ integration tests pass.
- [ ] Contract tests pass.
- [ ] Docker local integration works.
- [ ] Service builds independently.
- [ ] CI configuration is updated.
- [ ] Architecture diagrams are updated.
- [ ] No cross-service database dependencies were introduced.
- [ ] Completion report is generated.

Mark unavailable verification explicitly.

---

15. Completion Report

Create:

`agrosphere-architecture/docs/milestones/milestone-3b-completion.md`

Include:

1. Executive summary.
2. Verified Milestone 3A prerequisites.
3. Repositories modified.
4. Domain model implementation.

Take your curiosity further
Scan to get the ChatGPT app

5. REST endpoints implemented.
6. Database migrations.
7. Domain event contracts.
8. Security/authorization changes.
9. Automated test results.
10. DRC and reconciliation example calculations.
11. Docker integration results.
12. CI verification.
13. Architecture diagrams.
14. ADRs.
15. Known issues and technical debt.
16. Explicit scope exclusions.
17. Milestone 3C readiness.

Conclude with:

```
AGROSPHERE MILESTONE 3B COMPLETION REPORT

Status: COMPLETE / PARTIAL / BLOCKED

Repositories Changed:
...

Functional Capabilities:
...

REST APIs:
...

Domain Events:
...

Database Migrations:
...
```

Take your curiosity further
Scan to get the ChatGPT app

```
Security:
...

Unit Tests:
...

Integration Tests:
...

Contract Tests:
...

Docker Integration:
...

Architectural Decisions:
...

Outstanding Issues:
...

Readiness for Milestone 3C:
READY / NOT READY
```

---

16. Execution Phases

Execute the milestone in the following order.

Phase 1 — Discovery

Inspect actual code, 3A completion evidence, repositories, contracts, and infrastructure.

Phase 2 — Domain and Contract Design

Define collection, batch, weighment, DRC, production, reconciliation, and correction models. Agree on invariants, precision, ownership, and lifecycle transitions through implementation decisions documented in ADRs.

Take your curiosity further
Scan to get the ChatGPT app

Phase 3 — Database Implementation

Create safe Flyway migrations, persistence entities, repositories, constraints, and database tests.

Phase 4 — Business Services

Implement collection, batch, weighment, DRC calculation, production confirmation, and reconciliation workflows.

Phase 5 — API and Security

Implement REST endpoints, validation, DTOs, Keycloak authorization, estate-scoped access, and gateway integration.

Phase 6 — RabbitMQ Integration

Implement domain events, contracts, outbox publishing, and retry verification.

Phase 7 — Quality Engineering

Execute unit, integration, contract, and concurrency tests. Fix defects and verify build results.

Phase 8 — Documentation

Update PlantUML diagrams, contracts, ADRs, README files, and operational instructions.

Phase 9 — Completion

Generate the completion report, accurately distinguishing verified, unverified, and incomplete requirements.

Proceed autonomously through all phases.

Do not stop after architecture design or planning.

Do not begin Milestone 3C automatically.

Final objective: Deliver a fully implemented, auditable latex-production tracking workflow that integrates with the existing AgroSphere operations service without violating microservice boundaries.

Take your curiosity further
Scan to get the ChatGPT app

What Milestone 3B should deliver

At the end of this milestone, AgroSphere should be able to process a scenario like this:

Example production transaction

Collection source
Plantation Block B-12

Collection batch
LATEX-2026-001

Gross weight
285.500 kg

Tare weight
35.500 kg

Net latex weight
250.000 kg

Measured DRC
32.50%

CALCULATED DRY RUBBER

81.250 kg

250.000 kg × 32.50 ÷ 100

Example calculation valid

Illustrative values only; the calculation becomes an authoritative production result after the required measurements and approvals.

These production records and domain events will become inputs to Milestone 3E analytics and Milestone 3F CesiumJS visualization.

Next milestone: 3C — Crop Health Inspections, Disease Monitoring, Treatment Records, and Field Observations. This will introduce another important dimension to AgroSphere's digital twin: not just how much rubber the plantation produces, but the health and condition of the trees producing it.

Take your curiosity further
Scan to get the ChatGPT app

Take your curiosity further
Scan to get the ChatGPT app